

• EXPLORE · ATTACK · CONTAIN · REPORT

The Agent-Native Red-Teaming Workspace.

- ▶ Explore like a hacker. Your agent maps the app through its own browser.
- ▶ Attack the traffic. It edits, replays, and compares real requests.
- ▶ Stay in bounds. Sandbox limits you set, and a record of what it did.

Professional red-teaming for everyone.

■ THE PROBLEM

Professional red teaming is out of reach for most teams.

01 · Expertise

Security expertise is hard to find

Red teaming means understanding an app, forming attack hypotheses, and verifying them. Few teams have someone who does that regularly.

02 · Cost

Outside assessments are expensive

A professional pentest is valuable, but paying for one on every release is out of reach for many teams.

03 · Tooling

Existing tools were built for people

GUIs, proxies, and CA setup leave an agent reading screens and juggling tools instead of testing the app.

04 · Control

AI agents can run out of control

A mistake or a prompt injection can aim testing at the wrong systems. Teams need enforced limits and a record of what happened.

■ WHAT H5I GIVES YOU

Professional red-teaming for everyone.

No security expertise needed. Your own AI agent does the testing with h5i, and you review what it found.

h5i browser

Agent-led exploration

Your agent reads pages, logs in, clicks, and fills forms to map the app and its endpoints.

h5i websec

Direct HTTP attacks

It edits, replays, and compares the requests the browser sent, like a professional pentester.

h5i box · .h5i/env.toml

Safe execution

The agent works inside network and filesystem limits you set, and refused actions are recorded.

h5i ui · h5i project

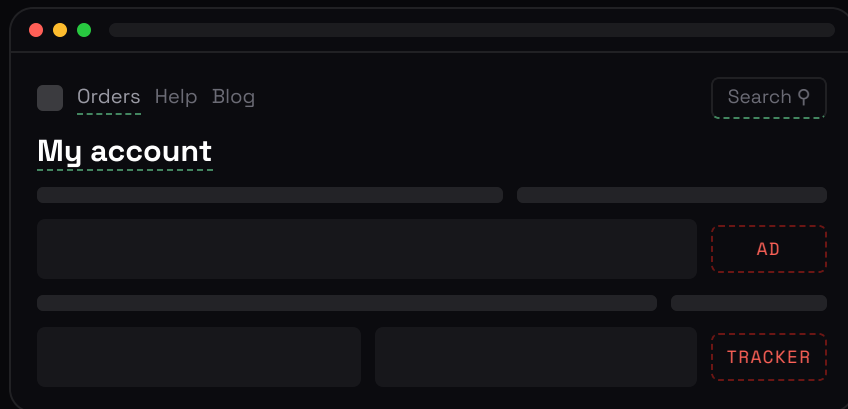
Results you can read

Review findings with their evidence, then share a report written for non-specialists.

■ THE BROWSER

Let the agent explore the app like a hacker.

THE TARGET · WHAT A PAGE LOADS



▶▶▶

PAGE BYTES

CAPTURES EVERY REQUEST

h5i browser

▶▶▶

OUTLINE, NOT PIXELS

THE AGENT · WHAT IT RUNS, AND WHAT COMES BACK

the agent's session

```
$ h5i browser open https://shop.test --capture
$ h5i browser snapshot
  @e1 heading "My account"
  @e7 link "Orders"
$ h5i browser click @e7 ✓
```

* Simple-site benchmarks vs headless Chromium · see the browser design notes

~3x

faster reads*

~86%

less peak memory*

@refs

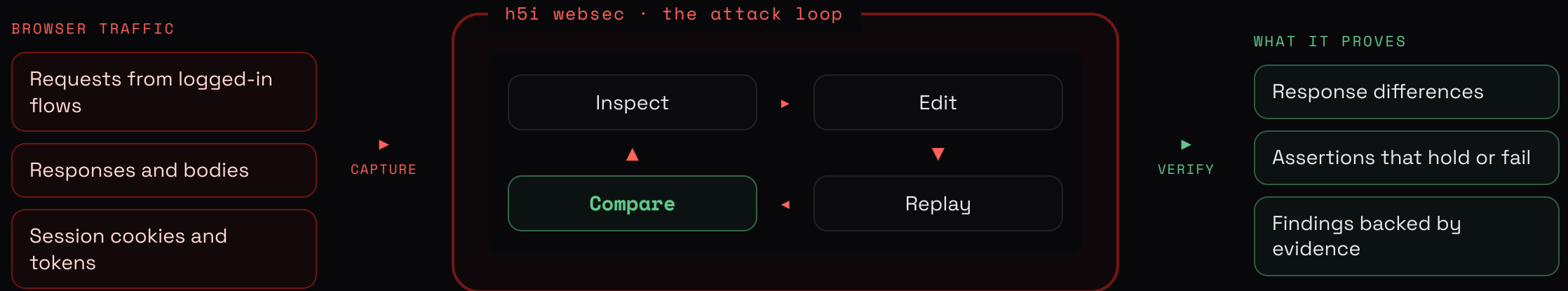
pages as outlines to act on

Pure Rust

no Chromium or V8

Let your agent red-team the web app.

The agent takes a request the app really sent, changes one field, resends it, and compares the answers. That is how it confirms what an attacker could actually do. No separate proxy, CA setup, or repeater.



Swap **id=123** for **id=456**. If another user's order comes back, that is a finding. Only on apps you are **authorized to test**.

Keep agent-led testing within bounds.

Browser policy covers browser traffic. To constrain the agent and its other tools, sandbox the whole workflow; enforcement depends on the isolation tier.

```
h5i browser open --allow
```

Limit where the browser goes

Browser policy rejects destinations outside the target and records each denied attempt.

```
h5i box · .h5i/env.toml
```

Isolate the whole workflow

Run the agent and its tools inside a sandbox with explicit network and filesystem limits.

```
h5i browser audit · requests
```

Record actions and refusals

Review browser actions, HTTP traffic, and denied connections with their reasons.

```
h5i ui
```

Review and intervene

Watch sessions live and take over sensitive browser steps when a human is needed.

■ WHY H5I

One workflow for agent-led red teaming. Fewer tools to coordinate.

Browser automation

The agent can drive pages. HTTP testing, limits, and findings live in other tools.

Browser + proxy tools

Deep traffic inspection built around a human GUI. The agent coordinates browser, proxy, and CA setup.

Sandboxing an existing browser

Limits what the agent can reach, without HTTP inspection, replay, or findings.

h5i

Exploration, the HTTP attack loop, sandbox limits, and reviewable findings in one CLI your agent already knows how to run.

Website compatibility remains a tradeoff: h5i is not a full Chromium replacement.

■ RESULTS & REPORTS

See what the agent found. Share what matters.

Findings outlive the browser session, and the report reads clearly for people who are not security specialists.

h5i project finding

Findings with evidence

Each finding cites the request and response behind it, with credentials removed. Severity, confidence, and fix status stay separate.

h5i ui · Report tab

A report anyone can read

Security terms open plain-language definitions, so the team that fixes the bug understands it.

h5i project report

Check, freeze, export

Broken references fail the check. An issued report is a versioned snapshot you can export as PDF.

Imported checklists track coverage as counts, never as a score.

■ TRACTION & INITIAL USERS

Built in the open.

Built for teams that test more often.

650+

GitHub stars

60+

forks

20+

contributors

35+

published releases

GitHub · checked September 26, 2026 · [Repository and releases](#)

Built for · product teams

No security specialist

Let their own agent test each release, then review the findings and the report.

Built for · security engineers

Test between assessments

Run agent-led checks more often, with the evidence to verify every claim.

Built for · pentesters & researchers

Automate the repetitive work

Hand recon and routine checks to an agent in a sandbox, for engagements and CTFs.

■ NEXT MILESTONES

Make professional red teaming accessible to every team.

Product

Test more apps, more easily

Broaden website compatibility and make agent-led verification simpler to run and review.

Adoption · planned

Work with design partners

Goal: 10 design partners testing their own apps with h5i, leading to the first paid pilots.

Open source

Build with the community

Keep h5i local-first and Apache 2.0, improved alongside contributors and security researchers.

Try it with your own AI agent: `curl -fsSL https://h5i.dev/install.sh | sh -s -- --websec`

h5i.dev

github.com/h5i-dev/h5i